

به نام خدا



درس نظریه زبان ها و ماشین ها

ترم ۴۰۴۲

پروژه پایانی

طراحی و پیاده سازی شبیه ساز ماشین تورینگ

استاد: دکتر میرحسین دزفولیان

دانشجویان:

زینب خلیلی پاکرو ۴۰۳۱۲۳۵۸۰۱۱

فاطمه شعبانی فرزانه ۴۰۳۱۲۳۵۸۰۲۲

۳	 مقدمه
۴	 معماری پروژه
۵	 Machines
۵	 copy_machine
۷	 Simulator
۷	 Constants
۸	 Parser
۱۰	 Head
۱۲	 State
۱۳	 Tape
۱۴	 Transition
۱۵	 TuringMachine
۱۹	 main
۱۹	 نحوه پیاده سازی تابع انتقال
۲۱	 تحلیل زمانی اجرای ماشین ها
۲۱	 تحلیل زمانی ماشین تورینگ
۲۱	 تحلیل زمانی پیاده سازی شبیه ساز
۲۱	 نمونه اجرای ماشین ها
۲۳	 پاسخ سوالات مطرح شده در سند با توجه به پیاده سازی خود
۲۶	 چالش های پیاده سازی
۲۶	 نتیجه گیری
۲۷	 لینک پروژه در گیتهاب

مقدمه

ماشین تورینگ یکی از بنیادی‌ترین مدل‌های محاسباتی در نظریه زبان‌ها و ماشین‌ها است که توسط آلن تورینگ معرفی شد و امروزه به‌عنوان مبنای مطالعه الگوریتم‌ها، قابلیت محاسبه و طراحی سیستم‌های محاسباتی شناخته می‌شود. این مدل با وجود ساختار ساده خود، قادر به شبیه‌سازی عملکرد هر الگوریتم قابل محاسبه بوده و نقش مهمی در علوم کامپیوتر نظری ایفا می‌کند.

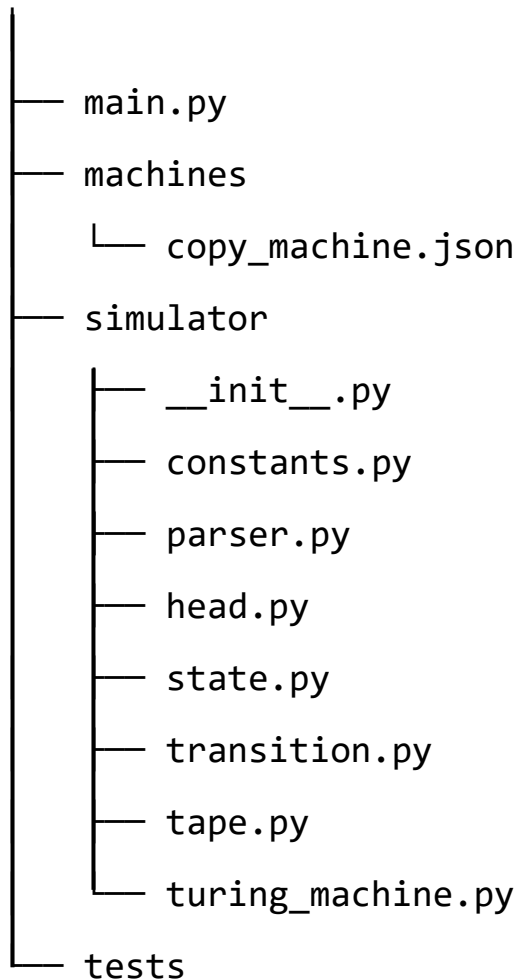
هدف از این پروژه، آشنایی عملی با نحوه عملکرد ماشین تورینگ از طریق طراحی و پیاده‌سازی یک شبیه‌ساز ماشین تورینگ و استفاده از آن برای اجرای یک ماشین نمونه است. در بخش نخست پروژه، یک شبیه‌ساز عمومی توسعه یافته است که قادر است مشخصات یک ماشین تورینگ را از فایل JSON دریافت کرده، صحت آن را بررسی نموده و ماشین را بر اساس تابع انتقال اجرا کند. این شبیه‌ساز از اجزایی مانند حالت‌ها، تابع انتقال، نوار، هد خواندن و نوشتن و مکانیزم توقف تشکیل شده و امکان اجرای هر ماشین تورینگ استاندارد را فراهم می‌کند.

در بخش دوم پروژه، یک ماشین تورینگ کپی (Copy Machine) طراحی و پیاده‌سازی شده است. این ماشین با دریافت رشته‌ای از نمادهای دودویی روی نوار، نسخه‌ای از آن را در محل مشخص شده تولید می‌کند. طراحی این ماشین به گونه‌ای انجام شده است که ضمن رعایت قوانین ماشین تورینگ، از حالت‌ها و انتقال‌های مناسب برای انجام عملیات کپی، بازگردانی نمادهای کمکی و خاتمه صحیح اجرای ماشین استفاده شود.

در این گزارش، ابتدا معماری کلی پروژه و ساختار فایل‌های آن معرفی شده، سپس نحوه پیاده‌سازی بخش‌های مختلف شبیه‌ساز، ساختمان داده‌های استفاده‌شده و فرآیند اجرای ماشین تشریح می‌شود. در ادامه، طراحی ماشین کپی، نحوه عملکرد آن، نتایج اجرای برنامه بر روی نمونه‌های آزمایشی و پاسخ سوالات مطرح‌شده در صورت پروژه ارائه خواهد شد.

معماری پروژه

Turing-machine-simulator



پروژه حاضر با هدف پیاده‌سازی یک شبیه‌ساز ماشین تورینگ و طراحی ماشین کپی (Copy Machine) به‌صورت ماژولار توسعه داده شده است. به‌منظور افزایش خوانایی، قابلیت توسعه و نگهداری آسان‌تر، هر بخش از سیستم در قالب فایل‌ها و پوشه‌های مجزا پیاده‌سازی شده است. ساختار کلی پروژه در شکل زیر نشان داده شده است.

➤ توضیح اجزای پروژه

فایل / پوشه	توضیح
main.py	نقطه شروع اجرای برنامه است و وظیفه دریافت ورودی، بارگذاری فایل ماشین، ایجاد اجزای اصلی شبیه‌ساز، مدیریت اجرای ماشین تورینگ و رسیدگی به خطاهای احتمالی را بر عهده دارد.
machines/	شامل فایل‌های تعریف ماشین تورینگ با فرمت JSON است. در این پروژه فایل copy_machine.json مشخصات ماشین کپی را نگهداری می‌کند.
simulator/	هسته اصلی شبیه‌ساز ماشین تورینگ است و کلاس‌های مربوط به پردازش فایل ماشین، نوار، حالت‌ها، توابع انتقال و اجرای ماشین در این پوشه قرار دارند.
tests/	شامل ورودی‌های آزمایشی برای بررسی عملکرد ماشین و ارزیابی صحت اجرای برنامه است.

➤ اجزای اصلی شبیه‌ساز

هسته اصلی برنامه در پوشه simulator قرار دارد. هر کلاس وظیفه مشخصی را بر عهده دارد و مجموع این کلاس‌ها فرآیند اجرای ماشین تورینگ را تشکیل می‌دهند. وظیفه هر کلاس به شرح زیر است:

کلاس	وظیفه
Parser	خواندن فایل JSON و تبدیل اطلاعات آن به اشیای State و Transition مورد استفاده در شبیه‌ساز.
Head	مدیریت موقعیت هد خواندن و نوشتن روی نوار و کنترل حرکت آن به سمت چپ یا راست.
State	نمایش اطلاعات هر حالت ماشین تورینگ، شامل نام حالت و مشخص بودن حالت پذیرش یا رد.
Transition	نمایش هر تابع انتقال شامل حالت فعلی، نماد خوانده‌شده، حالت بعدی، نماد نوشته‌شده و جهت حرکت هد.
Tape	مدیریت نوار ماشین تورینگ، شامل خواندن و نوشتن نمادها، توسعه پویا نوار و نگهداری محتوای آن.
TuringMachine	هسته اصلی شبیه‌ساز که با استفاده از حالت‌ها، توابع انتقال، نوار و هد، فرآیند اجرای ماشین تورینگ را مدیریت می‌کند.

علاوه بر کلاس‌های فوق، فایل constants.py نیز در پوشه simulator قرار دارد که مقادیر ثابت و تنظیمات مشترک مورد استفاده در بخش‌های مختلف شبیه‌ساز را در یک محل متمرکز نگهداری می‌کند.

Machines

copy_machine

copy_machine.json

```
— states
— input_alphabet
— tape_alphabet
— blank_symbol
— start_state
— accept_states
— reject_states
— transitions
```

هدف : فایل copy_machine.json توصیف کامل ماشین تورینگ

کپی را در قالب JSON نگهداری می‌کند. این فایل مستقل از پیاده‌سازی شبیه‌ساز بوده و تمامی اجزای موردنیاز ماشین شامل مجموعه حالت‌ها، الفبای ورودی، الفبای نوار، حالت شروع، حالت‌های پذیرش و رد و همچنین تابع انتقال را در خود ذخیره می‌کند. در زمان اجرای برنامه، این اطلاعات توسط کلاس Parser خوانده شده و به ساختارهای موردنیاز شبیه‌ساز تبدیل می‌شوند.

➤ ساختار کلی فایل

➤ معرفی حالت‌ها (States)

برای ساده‌تر شدن طراحی ماشین، عملیات کپی به چند مرحله مستقل تقسیم شده است. هر مرحله توسط گروه مشخصی از حالت‌ها انجام می‌شود. این روش باعث کاهش پیچیدگی تابع انتقال و افزایش خوانایی طراحی ماشین شده است.

گروه حالت	حالت‌ها	وظیفه
شروع	qStart	شروع اجرای ماشین و یافتن اولین نماد پردازش‌نشده
ذخیره نماد	qHave1, qHave0	ذخیره مقدار نماد پردازش شده در حالت ماشین و حرکت به انتهای خروجی
شیفت دروازه	qShift1, qShift0 qCarry1, qCarry0	انتقال های بلوک های X یک خانه به راست
نوشتن خروجی	qWrite1, qWrite0	نوشتن نماد در خروجی
بازگشت	qReturn	بازگشت هد به ابتدای نوار
بازیابی	qRestore	بازگردانی نمادهای کمکی A و B به حالت اولیه 0 و 1
پایان	qAccept, qFinish	حرکت تا رسیدن به نماد پایان G و خاتمه اجرای ماشین

➤ الفبای ماشین

ماشین تورینگ کپی از دو مجموعه الفبا استفاده می‌کند. الفبای ورودی تنها شامل نمادهای 0 و 1 است که داده اولیه ماشین را تشکیل می‌دهند. با این حال، برای اجرای صحیح الگوریتم کپی، استفاده از نمادهای کمکی نیز ضروری است. به همین منظور، الفبای نوار علاوه بر نمادهای ورودی، شامل نمادهای A و B برای علامت‌گذاری بیت‌های پردازش‌شده، نماد X برای نمایش ناحیه‌ای که در فرآیند جابه‌جایی (Shift) استفاده می‌شود، نماد # به‌عنوان جداکننده بین بخش ورودی و ناحیه کپی، نماد S به‌عنوان نشانگر ابتدای نوار، نماد G به‌عنوان مقصد نهایی هد پیش از پایان اجرا و نماد _ به‌عنوان فضای خالی نوار است. استفاده از این نمادهای کمکی موجب می‌شود ماشین بدون نیاز به هیچ حافظه خارجی، تنها بر اساس حالت فعلی و محتوای نوار، عملیات کپی را به‌صورت صحیح و مرحله‌به‌مرحله انجام دهد.

➤ تابع انتقال

تابع انتقال ماشین از مجموعه‌ای از قوانین تشکیل شده است که برای هر زوج (حالت فعلی، نماد خوانده‌شده)، حالت بعدی، نماد جدید و جهت حرکت هد را مشخص می‌کنند. در این پروژه تمام انتقال‌ها در قالب اشیای JSON ذخیره شده‌اند تا توسط شبیه‌ساز به‌صورت مستقیم خوانده شوند.



```
1 {  
2     "current_state": "qStart",  
3     "read": "0",  
4     "next_state": "qHave0",  
5     "write": "A",  
6     "move": "R"  
7 },
```

برای مثال این انتقال نشان می‌دهد که ماشین پس از مشاهده اولین بیت ۰، آن را با نماد کمکی A علامت‌گذاری کرده و با ورود به حالت qHave0 عملیات کپی آن بیت را آغاز می‌کند.

➤ روند عملکرد ماشین کپی

عملکرد ماشین کپی را می‌توان به چند مرحله متوالی تقسیم کرد. ماشین ابتدا نمادهای ورودی را به ترتیب پردازش و علامت‌گذاری می‌کند، سپس با حرکت به ناحیه خروجی نسخه‌ای از هر نماد را ایجاد کرده و پس از بازگشت به ابتدای نوار، این فرآیند را تا پایان ورودی تکرار می‌کند. در پایان نیز نمادهای کمکی حذف شده و ماشین در حالت پذیرش متوقف می‌شود.

شروع ➔ خواندن بیت ➔ علامت‌گذاری ➔ حرکت به # ➔ شیف‌ت X ➔ نوشتن نماد کپی شده ➔ بازگشت ➔ بازیابی ➔ Accept

Simulator

Constants

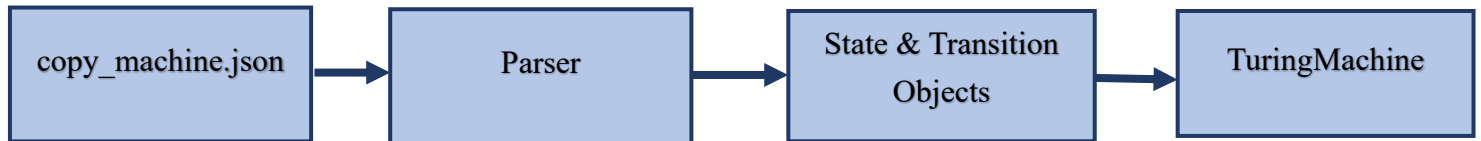
➤ هدف: فایل constants.py به منظور متمرکز کردن تمامی مقادیر ثابت مورد استفاده در شبیه‌ساز ماشین تورینگ ایجاد شده است. با قرار دادن این مقادیر در یک فایل مجزا، از تکرار آن‌ها در بخش‌های مختلف برنامه جلوگیری شده و خوانایی، نگهداری و توسعه کد بهبود یافته است. سایر بخش‌های پروژه مانند Head، Tape و TuringMachine در زمان اجرا از این مقادیر استفاده می‌کنند.

➤ ساختار داده های استفاده شده :

ساختار داده / کلاس	محل استفاده	دلیل استفاده
String	تعریف نماد خانه خالی و جهت حرکت هد و وضعیت های پایان اجرا	جلوگیری از استفاده از مقادیر ثابت (Magic Values) ، افزایش خوانایی و یکپارچگی کد
Integer	تعیین حداکثر تعداد گام های اجرای ماشین	جلوگیری از اجرای بی نهایت ماشین و کنترل زمان اجرای شبیه ساز

Parser

➤ هدف : Parser مسئول خواندن فایل توصیف ماشین تورینگ (با فرمت JSON) و تبدیل اطلاعات موجود در آن به اشیای قابل استفاده توسط شبیه ساز است. این کلاس نقش واسط بین فایل توصیف ماشین و شبیه ساز را بر عهده دارد و تمام اجزای ماشین شامل حالت ها، الفبای ورودی، الفبای نوار، حالت شروع و توابع انتقال را استخراج کرده و برای استفاده در کلاس TuringMachine آماده می کند.



روند تبدیل فایل توصیف ماشین تورینگ (JSON) به ساختارهای داده ای مورد استفاده در شبیه ساز توسط کلاس Parser

➤ ساختار داده و کلاس های استفاده شده :

ساختار داده / کلاس	محل استفاده	دلیل استفاده
dict[str, State]	ذخیره حالت های ماشین	دسترسی سریع به هر حالت با استفاده از نام آن
dict[(State, Symbol), Transition]	ذخیره توابع انتقال	بازیابی سریع انتقال متناظر با حالت و نماد خوانده شده
set	ذخیره الفبای ورودی و الفبای نوار	جلوگیری از تکرار و بررسی سریع عضویت
State	نمایش اطلاعات هر حالت	نگهداری نام حالت و مشخص کردن حالت پذیرش یا رد
Transition	نمایش یک تابع انتقال	نگهداری حالت فعلی، نماد خوانده شده، حالت بعدی، نماد نوشته شده و جهت حرکت

➤ توضیح توابع :

- تابع `__init__` : این تابع هنگام ایجاد شی `Parser` اجرا شده و وظیفه مقداردهی اولیه اعضای کلاس را بر عهده دارد. در این مرحله مسیر فایل ماشین ذخیره شده، بارگذاری می‌شود و ساختارهای داده موردنیاز برای ذخیره اطلاعات ماشین ایجاد می‌شوند.
- عملیات انجام شده**

```
1 def __init__(self, file_path: str):
2     """Initialize the parser."""
3
4     self.file_path = file_path
5     self.data = self.load_json()
6
7     self.input_alphabet = set(self.data["input_alphabet"])
8     self.tape_alphabet = set(self.data["tape_alphabet"])
9     self.blank_symbol = self.data["blank_symbol"]
10
11     self.states: dict[str, State] = {}
12     self.transitions: dict[tuple[str, str], Transition] = {}
13     self.start_state: State | None = None
```

- ذخیره مسیر فایل ماشین (`file_path`)
- فراخوانی تابع `load_json` و بارگذاری اطلاعات فایل
- استخراج الفبای ورودی (`input_alphabet`) و استخراج الفبای نوار (`tape_alphabet`)
- تعیین نماد فضای خالی (`blank_symbol`)
- ایجاد دیکشنری حالت‌ها (`states`) و ایجاد دیکشنری توابع انتقال (`transitions`)
- آماده‌سازی متغیر مربوط به حالت شروع (`start_state`)

- تابع `load_json` : این تابع فایل `JSON` را باز کرده و محتوای آن را به یک دیکشنری پایتون تبدیل می‌کند تا اطلاعات ماشین قابل پردازش باشد.

```
1 def load_json(self) -> dict:
2     """Load the JSON machine description."""
3
4     with open(self.file_path, "r", encoding="utf-8") as json_file:
5         return json.load(json_file)
```

- تابع `parse_states` : این تابع حالت‌های تعریف‌شده در فایل `JSON` را خوانده و برای هر حالت یک شی از کلاس `State` ایجاد کرده و در دیکشنری ذخیره می‌کند. همچنین مشخص می‌کند که هر حالت، عادی، پذیرش یا رد است.

```
1 for name in self.data["states"]:
2     self.states[name] = State(
3         name=name,
4         is_accept=name in accept_states,
5         is_reject=name in reject_states,
6     )
```

- تابع `parse_transitions` : تمام انتقال‌های موجود در فایل JSON خوانده شده و برای هر انتقال یک شی از کلاس `Transition` ساخته می‌شود. برای نگهداری انتقال‌ها از یک دیکشنری استفاده شده است که کلید آن به صورت `(Current State , Read Symbol)` است. به این ترتیب، هنگام اجرای ماشین، انتقال متناظر با حالت فعلی و نماد خوانده‌شده مستقیماً از دیکشنری بازیابی می‌شود و نیازی به جستجوی خطی در تمام انتقال‌ها نیست.

```
1 key = (  
2     transition.current_state,  
3     transition.current_symbol,  
4 )  
5  
6 self.transitions[key] = transition
```

- تابع `parse_machine` : این تابع مراحل مختلف تجزیه فایل را به ترتیب اجرا می‌کند. ابتدا حالت‌ها و انتقال‌ها ساخته می‌شوند، سپس حالت شروع تعیین شده و در نهایت تمام اجزای ماشین برای استفاده در کلاس `TuringMachine` بازگردانده می‌شوند.

➤ **الگوریتم کلی :** روند اجرای کلاس `Parser` به این صورت است که ابتدا فایل JSON ماشین تورینگ خوانده شده و اطلاعات آن به یک دیکشنری پایتون تبدیل می‌شود. سپس الفبای ورودی، الفبای نوار و نماد فضای خالی استخراج می‌شوند. در ادامه، برای هر حالت یک شی `State` و برای هر انتقال یک شی `Transition` ایجاد شده و انتقال‌ها در یک دیکشنری ذخیره می‌شوند. در نهایت، حالت شروع تعیین شده و تمامی اجزای موردنیاز ماشین برای استفاده در کلاس `TuringMachine` بازگردانده می‌شوند.

Head

- **هدف:** فایل `head.py` وظیفه پیاده‌سازی هد (Head) ماشین تورینگ را بر عهده دارد. هد ماشین مسئول خواندن و نوشتن روی نوار نیست، بلکه تنها موقعیت فعلی خود را روی نوار نگهداری کرده و بر اساس تابع انتقال، به سمت چپ یا راست حرکت می‌کند. همچنین در ابتدای اجرای هر ماشین، امکان بازنشانی (Reset) هد به موقعیت اولیه نیز فراهم شده است.

➤ ساختار داده های استفاده شده:

ساختار داده / کلاس	محل استفاده	دلیل استفاده
Class (Head)	پیاده سازی هد ماشین تورینگ	جداسازی وظیفه مدیریت هد از سایر بخش های برنامه و افزایش خوانایی کد
Constants	RIGHT و LEFT	جلوگیری از استفاده از مقادیر ثابت در کد و افزایش یکپارچگی برنامه

➤ توضیح توابع:

○ تابع `__init__`: این تابع هنگام ایجاد شیء `Head` فراخوانی می شود و موقعیت اولیه هد را برابر صفر قرار می دهد تا اجرای ماشین از ابتدای نوار آغاز شود.

○ تابع `move`: این تابع جهت حرکت هد را دریافت

کرده و بر اساس مقدار ورودی، یکی از توابع

`move_left()` یا `move_right()` را فراخوانی

می کند. همچنین اگر جهت حرکت معتبر نباشد، با

ایجاد خطا از اجرای نادرست برنامه جلوگیری

می کند.

```

1 def move(self, direction: str):
2
3     if direction == LEFT:
4         self.move_left()
5
6     elif direction == RIGHT:
7         self.move_right()
8
9     else:
10        raise ValueError("Invalid direction.")

```

○ تابع `move_left`: این تابع هد را یک خانه به سمت چپ حرکت می دهد و مقدار متغیر

`position` را یک واحد کاهش می دهد.

○ تابع `move_right`: این تابع هد را یک خانه به سمت راست حرکت داده و مقدار متغیر

`position` را یک واحد افزایش می دهد.

○ تابع `reset`: این تابع موقعیت هد را به مقدار اولیه (صفر) بازمی گرداند تا در صورت نیاز، اجرای

ماشین از ابتدای نوار آغاز شود.

➤ **الگوریتم کلی:** در ابتدای اجرای برنامه، موقعیت هد برابر صفر قرار می گیرد. در هر مرحله از اجرای

ماشین، تابع `move()` جهت حرکت را دریافت می کند. اگر جهت حرکت `LEFT` باشد، تابع

`move_left()` فراخوانی شده و موقعیت هد یک واحد کاهش می یابد. اگر جهت حرکت `RIGHT`

باشد، تابع `move_right()` اجرا شده و موقعیت هد یک واحد افزایش پیدا می کند. همچنین در

صورت دریافت جهت نامعتبر، برنامه با ایجاد خطا از ادامه اجرای نادرست جلوگیری می کند. در صورت

نیاز نیز با استفاده از تابع `reset()` هد دوباره به موقعیت اولیه بازگردانده می شود.

State

➤ هدف: فایل state.py وظیفه تعریف و مدیریت حالت‌های ماشین تورینگ را بر عهده دارد. هر شیء از کلاس State نمایانگر یک حالت از ماشین بوده و اطلاعاتی مانند نام حالت، پذیرشی (Accept) یا ردکننده (Reject) بودن آن را نگهداری می‌کند. این کلاس همچنین اعتبارسنجی اولیه حالت‌ها را انجام می‌دهد تا از ایجاد حالت‌های نامعتبر جلوگیری شود.

➤ ساختار داده‌های استفاده شده:

ساختار داده / کلاس	محل استفاده	دلیل استفاده
Class (State)	نمایش هر حالت ماشین تورینگ	کپسوله‌سازی اطلاعات مربوط به هر حالت و افزایش خوانایی برنامه
ValueError	اعتبارسنجی ورودی‌ها	جلوگیری از ایجاد حالت‌های نامعتبر

➤ توضیح توابع:

○ تابع `__init__`: این تابع هنگام ایجاد یک شیء از کلاس State فراخوانی می‌شود و مسئول مقداردهی اولیه ویژگی‌های حالت است. علاوه بر این، صحت اطلاعات ورودی را بررسی کرده و در صورت نامعتبر بودن نام حالت یا تعیین همزمان وضعیت پذیرش و رد، خطای مناسب ایجاد می‌کند.

```
1 def __init__(
2     self,
3     name: str,
4     is_accept: bool = False,
5     is_reject: bool = False,
6 ):
7
8     if not name:
9         raise ValueError("State name cannot be empty.")
10
11     if is_accept and is_reject:
12         raise ValueError("State cannot be both accept and reject.")
13
14     self.name = name
15     self.is_accept = is_accept
16     self.is_reject = is_reject
```

○ تابع `__repr__`: این تابع نحوه نمایش شیء State را مشخص می‌کند و با برگرداندن نام حالت، باعث خواناتر شدن خروجی برنامه و پیام‌های نمایش داده شده در زمان اجرای ماشین می‌شود.

Tape

➤ هدف: فایل `tape.py` وظیفه پیاده‌سازی نوار ماشین تورینگ را بر عهده دارد. این نوار محل ذخیره ورودی، داده‌های پردازش‌شده و نتایج اجرای ماشین است. در این پروژه، نوار با استفاده از ساختمان داده Dictionary پیاده‌سازی شده تا امکان گسترش نامحدود آن در هر دو جهت فراهم شود و تنها خانه‌های استفاده‌شده در حافظه ذخیره شوند.

➤ ساختار داده‌های استفاده شده:

ساختار داده / کلاس	محل استفاده	دلیل استفاده
Class (Tape)	پیاده‌سازی نوار ماشین تورینگ	مدیریت عملیات مربوط به نوار و جداسازی آن از سایر بخش‌های برنامه
Dictionary (cells)	ذخیره محتوای نوار	ذخیره فقط خانه‌های استفاده‌شده و امکان گسترش نامحدود نوار در هر دو جهت
List	تابع <code>render()</code>	جمع‌آوری نمادهای نوار برای تولید خروجی قابل نمایش

➤ توضیح توابع:

○ تابع `__init__`: این تابع نوار ماشین را مقداردهی اولیه می‌کند. ابتدا نوع ورودی بررسی می‌شود، سپس نمادهای رشته ورودی در دیکشنری `cells` ذخیره می‌شوند. اگر ورودی خالی باشد، تنها نماد خانه خالی در موقعیت صفر قرار می‌گیرد.

○ تابع `read`: این تابع نماد موجود در یک موقعیت مشخص از نوار را برمی‌گرداند. اگر آن موقعیت قبلاً

مقداردهی نشده باشد، نماد خانه خالی (`blank_symbol`) بازگردانده می‌شود.

○ تابع `write`: این تابع نماد جدید را در موقعیت مشخصی از نوار می‌نویسد. همچنین بررسی می‌کند که نماد ورودی تنها یک کاراکتر باشد و در غیر این صورت خطا ایجاد می‌کند.

```
1 def write(self, position: int, symbol: str) -> None:
2     if len(symbol) != 1:
3         raise ValueError("Tape symbols must be single characters.")
4
5     self.cells[position] = symbol
```

○ تابع `get_used_positions` : این تابع تمامی موقعیت‌هایی را که روی نوار مقداردهی شده‌اند، به صورت مرتب بازمی‌گرداند.

○ تابع `get_min_position` : این تابع کوچک‌ترین موقعیت استفاده‌شده روی نوار را محاسبه و بازمی‌گرداند.

○ تابع `get_max_position` : این تابع بزرگ‌ترین موقعیت استفاده‌شده روی نوار را محاسبه و بازمی‌گرداند.

```
1 def render(self, head_position: int) -> tuple[str, int]:
2
3     # Show negative positions only if they have actually been used
4     start = min(0, self.get_min_position())
5
6     # Show tape up to the furthest written cell or head position
7     end = max(head_position, self.get_max_position())
8
9     symbols = []
10
11     for position in range(start, end + 1):
12         symbols.append(self.read(position))
13
14     return " ".join(symbols), start
```

○ تابع `render` : این تابع وضعیت فعلی نوار را برای نمایش در خروجی آماده می‌کند. ابتدا بازه قابل نمایش نوار را تعیین کرده، سپس نمادهای موجود را جمع‌آوری می‌کند و در نهایت رشته‌ای از محتوای نوار به همراه اولین موقعیت نمایش‌داده‌شده را بازمی‌گرداند.

➤ **الگوریتم کلی:** در ابتدا نوار با استفاده از رشته ورودی مقداردهی اولیه می‌شود. هر نماد ورودی در یک خانه از دیکشنری ذخیره شده و در صورت خالی بودن ورودی، تنها نماد خانه خالی در موقعیت صفر قرار می‌گیرد. در طول اجرای ماشین، عملیات خواندن و نوشتن مستقیماً روی دیکشنری انجام می‌شود. همچنین در زمان نمایش وضعیت ماشین، تابع `render()` محتوای نوار را از کوچک‌ترین تا بزرگ‌ترین موقعیت استفاده‌شده استخراج کرده و به صورت یک رشته قابل نمایش تولید می‌کند.

Transition

➤ **هدف:** فایل `transition.py` وظیفه تعریف انتقال‌های ماشین تورینگ را بر عهده دارد. هر شیء از کلاس `Transition` نمایانگر یک قانون انتقال است که شامل حالت فعلی، نماد خوانده‌شده، حالت بعدی، نماد قابل نوشتن و جهت حرکت هد می‌باشد. این کلاس همچنین معتبر بودن جهت حرکت را بررسی می‌کند تا تنها مقادیر مجاز `LEFT` و `RIGHT` پذیرفته شوند.

➤ ساختار داده های استفاده شده:

ساختار داده / کلاس	محل استفاده	دلیل استفاده
Class (Transition)	نمایش هر انتقال ماشین تورینگ	کپسوله سازی اطلاعات مربوط به هر انتقال
Constants (LEFT, RIGHT)	بررسی مقدار direction	جلوگیری از ثبت جهت های نامعتبر و افزایش خوانایی کد

➤ توضیح توابع:

- تابع `__init__`: این تابع هنگام ایجاد یک شیء از کلاس Transition فراخوانی می شود. ابتدا مقدار جهت حرکت بررسی می شود تا تنها یکی از مقادیر LEFT یا RIGHT باشد. سپس اطلاعات مربوط به انتقال شامل حالت فعلی، نماد خوانده شده، حالت بعدی، نماد قابل نوشتن و جهت حرکت در ویژگی های شیء ذخیره می شود.

```

1 def __init__(
2     self,
3     current_state: str,
4     current_symbol: str,
5     next_state: str,
6     write_symbol: str,
7     direction: str,
8 ):
9
10     if direction not in (LEFT, RIGHT):
11         raise ValueError("Direction must be LEFT or RIGHT.")
12
13     self.current_state = current_state
14     self.current_symbol = current_symbol
15     self.next_state = next_state
16     self.write_symbol = write_symbol
17     self.direction = direction

```

- تابع `__repr__`: این تابع نحوه نمایش یک شیء Transition را مشخص می کند و اطلاعات انتقال را به صورت یک رشته خوانا برمی گرداند. این قابلیت در نمایش خروجی و اشکال زدایی برنامه بسیار مفید است.

TuringMachine

- هدف: فایل `turing_machine.py` هسته اصلی شبیه ساز ماشین تورینگ را تشکیل می دهد و مسئول مدیریت فرآیند اجرای ماشین است. این کلاس با استفاده از حالت های ماشین، تابع انتقال، نوار و هد، عملیات خواندن و نوشتن روی نوار، تغییر حالت، جابه جایی هد و بررسی شرایط پایان اجرا را انجام می دهد. همچنین اعتبارسنجی اولیه اطلاعات ماشین، نمایش وضعیت فعلی و اجرای مرحله به مرحله ماشین نیز در این فایل پیاده سازی شده است.

➤ ساختار داده های استفاده شده:

ساختار داده / کلاس	محل استفاده	دلیل استفاده
Class (TuringMachine)	پیاده سازی شبیه ساز ماشین تورینگ	مدیریت کل فرآیند اجرای ماشین
Dictionary (states)	ذخیره حالت های ماشین	دسترسی سریع به هر حالت با استفاده از نام آن
Dictionary (transitions)	ذخیره تابع انتقال	جستجوی سریع انتقال مناسب با کلید (State, Symbol)
Set (input_alphabet)	الفبای ورودی	بررسی معتبر بودن ورودی ها
Set (tape_alphabet)	الفبای نوار	اعتبارسنجی نمادهای قابل استفاده
Class (Tape)	مدیریت نوار ماشین	انجام عملیات خواندن و نوشتن
Class (Head)	مدیریت هد	کنترل موقعیت هد روی نوار
Class (State)	نگهداری وضعیت فعلی	مدیریت حالت جاری ماشین
Class (Transition)	نگهداری اطلاعات انتقال	اجرای قوانین انتقال

➤ توضیح توابع:

- تابع `__init__`: این تابع مسئول مقداردهی اولیه شبیه ساز و بررسی صحت اطلاعات ورودی است. در این بخش معتبر بودن حالت ها، الفباها، نماد خالی، تابع انتقال و تعداد گام های مجاز کنترل شده و سپس اجزای مختلف ماشین مقداردهی می شوند.

```

1 def __init__(
2     self,
3     states: dict[str, State],
4     transitions: dict[tuple[str, str], Transition],
5     start_state: State,
6     tape: Tape,
7     head: Head,
8     input_alphabet: set[str],
9     tape_alphabet: set[str],
10    max_steps: int = DEFAULT_MAX_STEPS,
11 ):
12     # Validation
13
14     if max_steps <= 0:
15         raise ValueError("max_steps must be greater than zero.")
16
17     if not states:
18         raise ValueError("States cannot be empty.")
19
20     if start_state.name not in states:
21         raise ValueError("Start state must exist in states.")
22
23     if not input_alphabet:
24         raise ValueError("Input alphabet cannot be empty.")
25
26     if not tape_alphabet:
27         raise ValueError("Tape alphabet cannot be empty.")
28
29     if tape.blank_symbol not in tape_alphabet:
30         raise ValueError("Blank symbol must belong to tape alphabet.")
31
32     # The input alphabet must be a subset of the tape alphabet
33     if not input_alphabet.issubset(tape_alphabet):
34         raise ValueError("Input alphabet must be a subset of tape alphabet.")

```

```

1 # Validate transitions
2 for transition in transitions.values():
3
4     if transition.current_state not in states:
5         raise ValueError(
6             f"Undefined current state: " f"{transition.current_state}"
7         )
8
9     if transition.next_state not in states:
10        raise ValueError(f"Undefined next state: " f"{transition.next_state}")
11
12    if transition.current_symbol not in tape_alphabet:
13        raise ValueError(
14            f"Invalid read symbol: " f"{transition.current_symbol}"
15        )
16
17    if transition.write_symbol not in tape_alphabet:
18        raise ValueError(f"Invalid write symbol: " f"{transition.write_symbol}")
19
20 # Store machine
21
22 self.states = states
23 self.transitions = transitions
24
25 self.input_alphabet = input_alphabet
26 self.tape_alphabet = tape_alphabet
27
28 self.current_state = start_state
29
30 self.tape = tape
31 self.blank_symbol = tape.blank_symbol
32
33 self.head = head
34
35 self.max_steps = max_steps
36 self.step_count = 0

```

- تابع `get_current_symbol`: نماد قرار گرفته زیر هد را از نوار خوانده و برای استفاده در فرآیند اجرای ماشین برمی گرداند.

```
1 def find_transition(self) -> Transition | None:
2
3     key = (
4         self.current_state.name,
5         self.get_current_symbol(),
6     )
7
8     return self.transitions.get(key)
```

- تابع `find_transition`: با استفاده از حالت فعلی و نماد خوانده شده، کلید مربوط به تابع انتقال ساخته شده و انتقال مناسب از دیکشنری انتقال ها استخراج می شود.

- تابع `step`: این تابع یک مرحله از اجرای ماشین را انجام می دهد. ابتدا انتقال مناسب پیدا می شود، سپس نماد جدید روی نوار نوشته شده، هد حرکت می کند، حالت فعلی تغییر یافته و شمارنده مراحل افزایش می یابد.

```
1 def step(self) -> bool:
2
3     transition = self.find_transition()
4
5     if transition is None:
6         return False
7
8     print("Transition:")
9     print(
10         f"({transition.current_state}, "
11         f"{transition.current_symbol}) "
12         f"--> "
13         f"({transition.next_state}, "
14         f"{transition.write_symbol}, "
15         f"{transition.direction})"
16     )
17     print()
```

```
1 # Write symbol
2 self.tape.write(
3     self.head.position,
4     transition.write_symbol,
5 )
6
7 # Move head
8 self.head.move(transition.direction)
9
10 # Change state
11 self.current_state = self.states[transition.next_state]
12
13 self.step_count += 1
14
15 return True
```

- تابع `is_accept`: بررسی می کند که آیا ماشین در حالت پذیرش قرار گرفته است یا خیر.
- تابع `is_reject`: بررسی می کند که آیا ماشین در حالت رد قرار گرفته است یا خیر.
- تابع `has_timed_out`: تعداد مراحل اجرا را با مقدار مجاز مقایسه کرده و در صورت عبور از حد تعیین شده، پایان اجرا را اعلام می کند.
- تابع `display_configuration`: وضعیت فعلی ماشین شامل شماره مرحله، حالت جاری، محتوای نوار، موقعیت هد و اندیس های نوار را به صورت خوانا نمایش می دهد.

```

1 def display_configuration(self) -> None:
2
3     tape_string, start_position = self.tape.render(self.head.position)
4
5     symbols = tape_string.split()
6
7     print(f"Step: {self.step_count}")
8     print(f"State: {self.current_state.name}")
9     print()
10
11     print("Index:")
12
13     indices = " ".join(
14         str(i) for i in range(start_position, start_position + len(symbols))
15     )
16
17     print(indices)
18
19     print("Tape:")
20     print(tape_string)

```

```

1 head_offset = self.head.position - start_position
2
3     pointer = []
4
5     for i in range(len(symbols)):
6
7         if i == head_offset:
8             pointer.append("^")
9
10        else:
11            pointer.append(" ")
12
13    print(" ".join(pointer))
14    print()
15
16    print(f"Head Position: {self.head.position}")
17
18    print("-" * 40)

```

○ تابع `get_status`: وضعیت فعلی ماشین را بررسی کرده و مشخص می‌کند که ماشین در حالت پذیرش، رد یا پایان زمان اجرا قرار دارد یا خیر.

```

1 def run(self) -> str:
2
3     while True:
4
5         self.display_configuration()
6
7         status = self.get_status()
8
9         if status is not None:
10
11             print(f"Machine {status}.")
12
13             return status
14
15         if not self.step():
16
17             print("No valid transition found.")
18
19             print("Machine Rejected.")
20
21             return REJECT
22

```

○ تابع `run`: تابع اصلی اجرای ماشین است. این تابع به صورت پیوسته وضعیت ماشین را نمایش داده، شرایط پایان اجرا را بررسی می‌کند و در صورت امکان، مرحله بعدی را اجرا می‌کند. این روند تا رسیدن به یکی از حالات توقف ادامه پیدا می‌کند.

➤ **الگوریتم کلی**: ابتدا اطلاعات مربوط به حالت‌ها، انتقال‌ها، الفبای ورودی، الفبای نوار، هد و نوار دریافت و اعتبارسنجی می‌شوند. سپس ماشین در حالت شروع قرار گرفته و در هر مرحله، نماد زیر هد خوانده می‌شود. بر اساس حالت فعلی و نماد خوانده‌شده، انتقال مناسب از جدول انتقال استخراج می‌شود. پس از آن، نماد جدید روی نوار نوشته شده، هد در جهت مشخص‌شده حرکت می‌کند و حالت ماشین تغییر می‌یابد. این فرآیند تا رسیدن به یکی از حالات پذیرش، رد یا پایان تعداد مجاز گام‌ها ادامه پیدا می‌کند.

main

- **هدف :** فایل `main.py` نقطه شروع اجرای برنامه است و وظیفه راه‌اندازی شبیه‌ساز ماشین تورینگ را بر عهده دارد. در این فایل ابتدا فایل توصیف ماشین (`copy_machine.json`) توسط کلاس `Parser` خوانده می‌شود، سپس ورودی اولیه از پوشه `tests` دریافت شده و اجزای اصلی شبیه‌ساز شامل نوار (`Tape`)، هد (`Head`) و ماشین تورینگ (`TuringMachine`) ایجاد می‌شوند. در نهایت، اجرای ماشین آغاز شده و امکان اجرای چندین تست به صورت متوالی نیز فراهم می‌شود.
- **الگوریتم کلی :** روند اجرای برنامه در فایل `main.py` به این صورت است که ابتدا نام فایل ورودی از کاربر دریافت می‌شود. سپس کلاس `Parser` فایل `JSON` را پردازش کرده و اطلاعات مربوط به حالت‌ها، انتقال‌ها، الفبای ورودی، الفبای نوار و حالت شروع را استخراج می‌کند. پس از آن، رشته ورودی از فایل تست خوانده شده و نوار اولیه ایجاد می‌شود. در ادامه، هد ماشین و شی `TuringMachine` ساخته شده و با فراخوانی تابع `run()`، اجرای ماشین آغاز می‌شود. پس از پایان اجرا، کاربر می‌تواند در صورت نیاز تست دیگری را اجرا کند.
- **مدیریت خطا :** برای افزایش پایداری برنامه، از ساختار `try-except` استفاده شده است. در صورتی که فایل ورودی وجود نداشته باشد، داده‌های ورودی نامعتبر باشند یا هر خطای پیش‌بینی نشده دیگری رخ دهد، پیام مناسب به کاربر نمایش داده شده و از خاتمه ناگهانی برنامه جلوگیری می‌شود.

نحوه پیاده سازی تابع انتقال

تابع انتقال ماشین تورینگ در این پروژه به صورت مستقیم در کد تعریف نشده است، بلکه قوانین انتقال در فایل `copy_machine.json` ذخیره شده‌اند. هر قانون انتقال شامل پنج مولفه اصلی است:

- حالت فعلی (`current_state`)
- نماد خوانده شده (`read`)
- نماد قابل نوشتن (`write`)
- جهت حرکت هد (`move`)
- حالت بعدی (`next_state`)

در زمان بارگذاری ماشین، کلاس `Parser` فایل `JSON` را پردازش کرده و برای هر قانون، یک شی از کلاس `Transition` ایجاد می‌کند. سپس تمامی انتقال‌ها در یک دیکشنری ذخیره می‌شوند که کلید آن به صورت

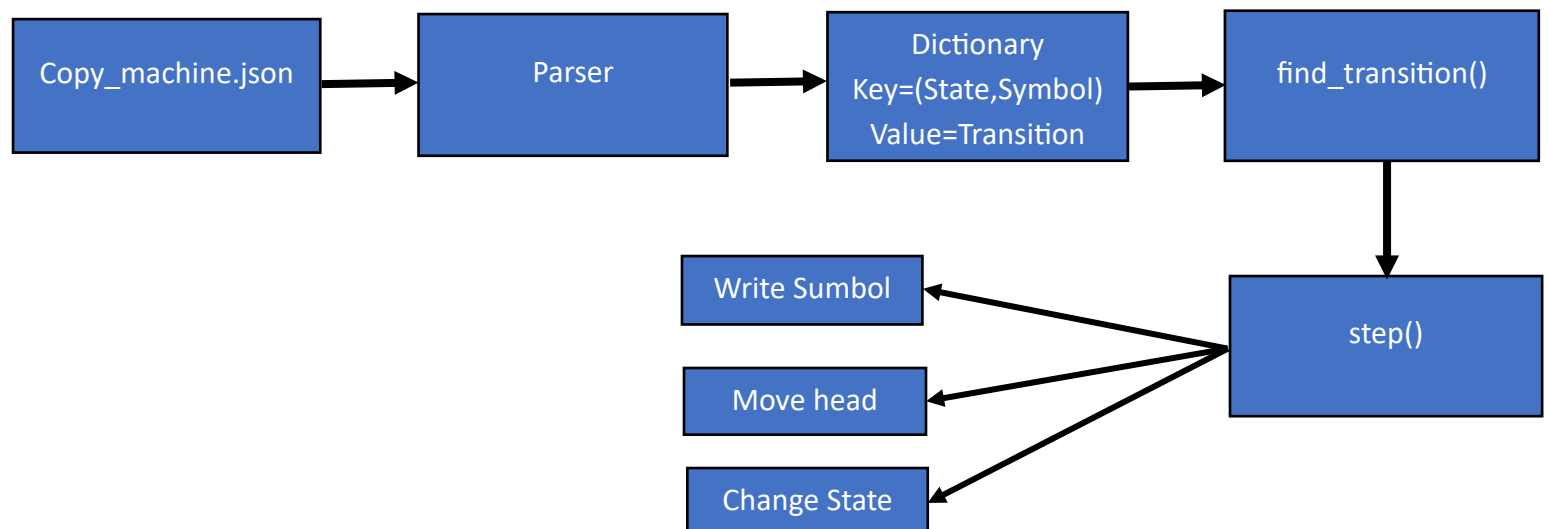
(Current State, Read Symbol) و مقدار آن شی Transition است.

این ساختار امکان دسترسی مستقیم به انتقال متناظر را با استفاده از حالت فعلی و نماد خوانده شده فراهم می کند و نیاز به پیمایش تمامی انتقال ها را از بین می برد. به عنوان نمونه، انتقال زیر در فایل JSON :

{	Key:
"current_state": "qStart",	(qStart, 1)
"read": "1",	Value:
"next_state": "qHave1",	(next_state=qHave1,
"write": "B",	write_symbol=B,
"move": "R"	direction=R)
}	

در دیکشنری به شکل بالا ذخیره می شود.

در زمان اجرای ماشین، کلاس TuringMachine ابتدا حالت فعلی و نماد زیر هد را دریافت کرده و با استفاده از تابع find_transition() انتقال متناظر را از دیکشنری بازیابی می کند. در صورت وجود انتقال، تابع step() نماد جدید را روی نواری می نویسد، هد را در جهت مشخص شده حرکت می دهد و حالت فعلی را به حالت بعدی تغییر می دهد. این فرآیند تا رسیدن ماشین به یکی از حالات توقف (پذیرش یا رد) ادامه پیدا می کند. همچنین اگر برای حالت فعلی و نماد خوانده شده انتقالی تعریف نشده باشد، ماشین اجرای خود را متوقف کرده و نتیجه Reject اعلام می شود.



تحلیل زمانی اجرای ماشین ها

تحلیل زمانی ماشین تورینگ

در ماشین کپی طراحی شده، برای هر نماد از رشته ورودی ابتدا نماد مربوطه از بخش ورودی خوانده شده و سپس هد ماشین تا انتهای نوار حرکت می کند تا نسخه ای از آن نماد را در قسمت خروجی ثبت نماید. پس از آن، هد مجدداً به ابتدای نوار بازمی گردد تا نماد بعدی پردازش شود. این فرآیند تا زمانی که تمام نمادهای ورودی پردازش شوند، تکرار می شود.

اگر طول رشته ورودی را n در نظر بگیریم، در هر تکرار هد مسافتی متناسب با طول نوار را طی می کند که دارای پیچیدگی زمانی $O(n)$ است. از آنجا که این عملیات برای تمام n نماد ورودی انجام می شود، پیچیدگی زمانی کل اجرای ماشین برابر است با $O(n^2)$. بنابراین، زمان اجرای ماشین با مجذور طول رشته ورودی رشد می کند که علت آن رفت و برگشت مکرر هد روی نوار برای پردازش هر نماد است.

تحلیل زمانی پیاده سازی شبیه ساز

در پیاده سازی انجام شده، تابع انتقال ماشین با استفاده از ساختار داده Dictionary ذخیره شده است. در نتیجه، یافتن انتقال مناسب بر اساس حالت فعلی و نماد خوانده شده به طور میانگین دارای پیچیدگی زمانی $O(1)$ است. همچنین عملیات خواندن و نوشتن روی نوار نیز به دلیل استفاده از ساختار مناسب برای ذخیره اطلاعات، در زمان ثابت انجام می شوند. بنابراین سربار ناشی از پیاده سازی شبیه ساز بسیار کم بوده و عامل اصلی زمان اجرای برنامه، تعداد مراحل اجرای خود ماشین تورینگ است.

نمونه اجرای ماشین ها

به منظور ارزیابی عملکرد شبیه ساز، ماشین تورینگ بر روی چند ورودی معتبر و نامعتبر اجرا شد. در این آزمایش ها، صحت عملکرد ماشین در پردازش رشته های مختلف، تولید خروجی صحیح و همچنین تشخیص ورودی های نامعتبر مورد بررسی قرار گرفت. در ادامه، نتایج اجرای تعدادی از تست های اصلی ارائه شده است.

```

-----
Transition:
(qFinish, G) --> (qAccept, G, L)

Step: 22
State: qAccept

Index:
0 1 2 3 4 5
Tape:
S 1 # 1 X G
      ^

Head Position: 4
-----
Machine ACCEPT.

```

۱. تست اول (ورودی معتبر کوتاه)

Input : S1#X_G

ماشین ورودی را با موفقیت پردازش کرده، عملیات کپی را انجام داده و در حالت پذیرش (Accept) متوقف شده است.

```

-----
Transition:
(qFinish, G) --> (qAccept, G, L)

Step: 63
State: qAccept

Index:
0 1 2 3 4 5 6 7 8 9 10
Tape:
S 1 0 # 1 0 X X X _ G
                ^

Head Position: 9
-----
Machine ACCEPT.

```

۲. تست دوم (ورودی با طول متوسط و مثال سند پروژه)

Input : S10#XXX__G

ماشین تمامی مراحل شامل علامت گذاری، جابه جایی بلوک های X، تولید نسخه کپی و بازگردانی نمادهای کمکی را به درستی انجام داده و در حالت پذیرش خاتمه یافته است.

```

-----
Transition:
(qReturn, A) --> (qReturn, A, L)

Step: 500
State: qReturn

Index:
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34
Tape:
S B A B B A B A B 0 1 # 1 0 1 1 0 1 0 0 1 X X X X _ _ _ _ _ G
                ^

Head Position: 4
-----
Machine TIMEOUT.

```

۳. تست سوم (ورودی بلند)

Input :

S10110100101#XXXXX_____G

در این آزمایش، به دلیل طولانی بودن فرآیند اجرای ماشین، تعداد گام های اجرا از مقدار مجاز تعیین شده (۵۰۰ گام) فراتر رفت. مطابق پیاده سازی انجام شده، شبیه ساز برای جلوگیری از اجرای نامحدود ماشین، پس از رسیدن به این آستانه اجرای ماشین را متوقف کرده و وضعیت TIMEOUT را گزارش می کند. این نتیجه نشان می دهد مکانیزم کنترل تعداد گام ها مطابق صورت پروژه به درستی عمل می کند.

```

-----
Transition:
(qReturn, B) --> (qReturn, B, L)

Step: 17
State: qReturn

Index:
0 1 2 3 4 5 6 7
Tape:
B 0 # 1 X X _ G

Head Position: -1
-----
No valid transition found.
Machine Rejected.

```

۴. تست چهارم (ورودی نامعتبر)

Input : 10#XX_G

از آنجا که ورودی فاقد نماد شروع S است، ورودی نامعتبر تشخیص داده شده و اجرای ماشین آغاز نمی شود. پیام خطای مناسب به کاربر نمایش داده می شود که نشان دهنده عملکرد صحیح بخش اعتبارسنجی ورودی است.

پاسخ سوالات مطرح شده در سند با توجه به پیاده سازی خود

۱. چرا PDA نمی تواند این مسئله را حل کند؟ کجای طراحی TM شما بود که PDA شکست می خورد؟

در بخش دوم پروژه، ماشین تورینگ وظیفه دارد کد دسترسی را از روی نوار خوانده، آن را پس از نماد # کپی کند و همزمان فضای لازم را با جابه جایی نمادهای X ایجاد نماید. در پیاده سازی ما، هر بیت پس از خوانده شدن با استفاده از نمادهای کمکی A و B علامت گذاری می شود. سپس ماشین با استفاده از حالت هایی مانند qHave0, qHave1, qShift0, qShift1, qCarry0, qCarry1, qReturn و qRestore چندین بار بین ابتدای رشته و انتهای نوار حرکت می کند، نمادها را بازنویسی می کند و در پایان رشته اصلی را بازیابی می کند.

ماشین پشته ای (PDA) تنها از یک پشته برای ذخیره اطلاعات استفاده می کند و امکان بازنویسی آزادانه نوار و حرکت رفت و برگشتی روی ورودی را ندارد. در حالی که ماشین طراحی شده در این پروژه برای هر بیت چندین بار روی نوار حرکت کرده، اطلاعات را تغییر می دهد و نسخه جدید رشته را ایجاد می کند. بنابراین قابلیت های PDA برای انجام این عملیات کافی نیست و استفاده از ماشین تورینگ ضروری است.

۲. آیا می شد توصیف ماشین تورینگ را با یک گرامر نوشت؟

بله. از دیدگاه نظری، هر زبانی که توسط ماشین تورینگ قابل شناسایی باشد، با یک گرامر نوع صفر (Type-0) نیز قابل توصیف است. بنابراین رفتار ماشین تورینگ را می توان به صورت مجموعه ای از قواعد بازنویسی بیان کرد.

با این حال، در این پروژه برای افزایش خوانایی و توسعه پذیری، ماشین به صورت یک فایل JSON پیاده سازی شده است. فایل copy_machine.json شامل مجموعه حالت ها، الفباها، حالت شروع، حالت های پذیرش و رد و همچنین تمامی انتقال های ماشین است. کلاس Parser این اطلاعات را خوانده و آن ها را به اشیای State و Transition تبدیل می کند. به این ترتیب منطق ماشین از کد برنامه جدا شده و بدون تغییر در شبیه ساز، می توان ماشین های مختلف را اجرا کرد.

۳. پیچیدگی زمانی و فضایی پیاده‌سازی شما در بخش دوم از چه مرتبه‌ای بود؟

در بخش دوم پروژه، برای کپی کردن کد دسترسی، ماشین تورینگ هر بیت ورودی را به ترتیب پردازش می‌کند. در این فرآیند، هر بیت ابتدا با نمادهای کمکی A یا B علامت‌گذاری شده، سپس ماشین تا انتهای نوار حرکت می‌کند، در صورت نیاز بلوک‌های X را جابه‌جا می‌کند، بیت را در محل جدید می‌نویسد و دوباره به ابتدای رشته بازمی‌گردد تا بیت بعدی را پردازش کند. از آنجا که این عملیات برای هر یک از n بیت‌های ورودی روی بخش قابل توجهی از نوار تکرار می‌شود، پیچیدگی زمانی الگوریتم از مرتبه $O(n^2)$ است. همچنین، به دلیل اینکه علاوه بر رشته اصلی، یک نسخه از آن روی نوار ایجاد می‌شود و تنها از چند نماد کمکی مانند A و B نیز استفاده می‌شود، میزان فضای مصرفی متناسب با طول ورودی بوده و پیچیدگی فضایی برابر $O(n)$ است.

- با توجه به اجرای خودتان، هاتعداد تقریبی گام (تعداد انتقال) برای کپی کردن یک کد با طول n چقدر است؟ با توجه به اینکه برای پردازش هر بیت چندین حرکت رفت و برگشتی روی نوار انجام می‌شود، تعداد انتقال‌های ماشین تقریباً از مرتبه $O(n^2)$ است. بنابراین با افزایش طول ورودی، تعداد گام‌های اجرا نیز به صورت درجه دوم افزایش پیدا می‌کند.
- اگر به جای ماشین تورینگ تک‌نواره، از ماشین تورینگ چندنواره استفاده می‌کردید، آیا پیچیدگی زمانی کاهش می‌یافت؟ چگونه؟ بله. در ماشین تورینگ چندنواره، رشته ورودی می‌تواند روی نوار اول باقی بماند و هم‌زمان روی نوار دوم کپی شود. در نتیجه دیگر نیازی به حرکت‌های مکرر بین ابتدای نوار و محل کپی و همچنین جابه‌جایی مداوم اطلاعات روی یک نوار نیست. به همین دلیل تعداد حرکت‌های اضافی کاهش یافته و پیچیدگی زمانی از $O(n^2)$ به $O(n)$ کاهش می‌یابد، در حالی که پیچیدگی فضایی همچنان از مرتبه $O(n)$ باقی می‌ماند.

۴. بر اساس تجربه پیاده‌سازی خودتان، مشخصاً کدام بخش از پروژه نیاز به حافظه نامحدود (بیش از یک

پشته) داشت و چرا؟

بیشترین نیاز به حافظه نامحدود در مرحله کپی کردن کد دسترسی وجود داشت. در این مرحله ماشین باید رشته اصلی را حفظ کند، بیت‌های پردازش‌شده را با A و B علامت‌گذاری کند، بلوک‌های X را به سمت راست شیفت دهد، نسخه جدید رشته را پس از نماد $\#$ بنویسد و سپس دوباره به ابتدای نوار بازگردد تا بیت بعدی را پردازش کند. انجام هم‌زمان این عملیات تنها با استفاده از یک پشته امکان‌پذیر نیست، زیرا PDA قادر به بازنویسی نوار و حرکت آزاد هد روی آن نیست. اما ماشین تورینگ با داشتن

نواری با حافظه نامحدود و امکان خواندن، نوشتن و حرکت در هر دو جهت، این عملیات را به درستی انجام می دهد.

- اگر به جای کپی کردن کد، فقط لازم بود بررسی کنیم آیا کد دسترسی پالیندروم است، آیا TM لازم بود یا PDA کافی بود؟ چرا؟ اگر تنها هدف تشخیص پالیندروم بودن رشته باشد، در حالت معمول PDA کافی است؛ زیرا می توان نیمه اول رشته را در پشته ذخیره کرد و سپس آن را با نیمه دوم مقایسه نمود. اما پروژه حاضر تنها یک مسئله تشخیص نیست، بلکه نیازمند بازنویسی نوار، جابه جایی بلوک های X، کپی کردن رشته و حرکت های مکرر هد است. بنابراین برای این پروژه استفاده از ماشین تورینگ ضروری است.

۵. این مسئله کجای سلسله مراتب چامسکی قرار می گیرد؟

ماشین طراحی شده در این پروژه از قابلیت های کامل ماشین تورینگ مانند حافظه نامحدود، حرکت آزاد هد روی نوار و بازنویسی اطلاعات استفاده می کند. از آنجا که انجام عملیات کپی کردن رشته و جابه جایی نمادها به این قابلیت ها نیاز دارد، این مسئله در مدل محاسباتی ماشین تورینگ بررسی می شود.

اگر زبان متناظر با این محاسبات را در نظر بگیریم، این زبان در رده زبان های نوع صفر (Type-0) یا زبان های بازگشتی قابل شمارش (Recursively Enumerable Languages) قرار می گیرد. بنابراین مسئله حاضر در سلسله مراتب چامسکی در سطح Type-0 قرار می گیرد، زیرا قابلیت هایی فراتر از ماشین متناهی (FA) و ماشین پشته ای (PDA) نیاز دارد و تنها با استفاده از ماشین تورینگ قابل پیاده سازی است.

چالش های پیاده سازی

در فرآیند طراحی و پیاده سازی این پروژه، چالش های مختلفی وجود داشت که مهم ترین آن ها مربوط به طراحی منطق ماشین تورینگ و هماهنگی اجزای شبیه ساز بود. یکی از چالش های اصلی، مدیریت موقعیت هد روی نوار بود؛ به گونه ای که در هر مرحله محل دقیق هد مشخص باشد و تغییرات آن به درستی در خروجی نمایش داده در ماشین کپی بود. این بخش X نمادهای (Shift) شود. چالش مهم دیگر، پیاده سازی فرآیند جابه جایی به صورت صحیح X نیازمند طراحی دقیق حالت ها و انتقال ها بود تا بدون از بین رفتن اطلاعات، بلوک نمادهای جابه جا شده و فضای لازم برای نوشتن خروجی ایجاد شود.

همچنین، هماهنگی بین فایل JSON و ساختار داخلی برنامه از اهمیت بالایی برخوردار بود؛ زیرا تمامی حالت ها و انتقال ها باید بدون خطا توسط کلاس Parser خوانده شده و به ساختارهای مورد استفاده در شبیه ساز تبدیل می شدند.

از دیگر چالش ها، اعتبارسنجی ورودی ها و مدیریت خطاها بود. برنامه باید بتواند در صورت وجود ورودی نامعتبر، نبود فایل تست، یا تعریف نشدن انتقال مناسب، اجرای خود را به صورت کنترل شده متوقف کرده و پیام خطایی مناسب نمایش دهد.

در نهایت، هماهنگ سازی اجزای مختلف شبیه ساز شامل نوار، هد، حالت ها، انتقال ها و موتور اجرای ماشین، به گونه ای که در هر گام عملیات خواندن، نوشتن، تغییر حالت و حرکت هد مطابق تعریف ماشین تورینگ انجام شود، از مهم ترین بخش های پیاده سازی این پروژه بود.

نتیجه گیری

در این پروژه، یک شبیه ساز ماشین تورینگ مطابق مشخصات ارائه شده در صورت پروژه طراحی و پیاده سازی شد. این شبیه ساز با دریافت مشخصات ماشین از فایل JSON، ایجاد اجزای اصلی شامل حالت ها، تابع انتقال، نوار و هد، امکان اجرای ماشین های تورینگ را به صورت عمومی فراهم می کند. همچنین یک ماشین کپی به عنوان نمونه طراحی شد که قادر است رشته ورودی را مطابق الگوریتم تعریف شده روی نوار کپی کرده و در پایان به حالت پذیرش برسد.

نتایج اجرای برنامه بر روی مجموعه ای از ورودی های معتبر و نامعتبر نشان داد که شبیه ساز علاوه بر انجام صحیح عملیات کپی، ورودی های نامعتبر را نیز به درستی تشخیص می دهد و محدودیت تعداد گام های اجرا را

برای جلوگیری از اجرای نامتناهی اعمال می‌کند. طراحی ماژولار پروژه، جداسازی مسئولیت کلاس‌ها و استفاده از فایل JSON برای تعریف ماشین، باعث افزایش خوانایی، توسعه‌پذیری و امکان استفاده مجدد از شبیه‌ساز برای ماشین‌های تورینگ دیگر شده است.

لینک پروژه در گیت‌هاب

<https://github.com/Ftmshb/turing-machine-simulator.git>